SUMANA SOMINENI

APPLIED AI ENGINEER | GENERATIVE AI & AGENTIC SYSTEMS | FULL-STACK DEVELOPMENT

Overland Park, KS | Open to relocation | 859-452-2920

sumanasomineni0909@gmail.com | LinkedIn | GitHub | Portfolio

PROFESSIONAL SUMMARY

AI Engineer with 4+ years building, evaluating, and deploying production AI systems for healthcare and education platforms serving 50K+ users. Built an AI document-extraction pipeline on Amazon Bedrock that cut referral processing from 40 minutes to 7 minutes per document at 94% field-level accuracy, and shipped a production computer vision service at ~95% recognition accuracy across 9 healthcare facilities. Designed and ran 1,000+ controlled LLM experiments comparing deployment and orchestration architectures across local and cloud inference environments, and co-authored the resulting comparative paper (in preparation).

PROFESSIONAL EXPERIENCE

AI Software Developer Jan 2026 – Present

CariCanary – Healthcare SaaS Platform *Overland Park, KS*

- Eliminated hand-recorded activity attendance across 9 healthcare facilities by building a facial-recognition service in Python/FastAPI (InsightFace, ArcFace, OpenCV, ONNX Runtime) that identifies residents in group photographs and flags unknown individuals — ~95% accuracy across ~1,000 test face instances at ~2s end-to-end inference.
- Improved recognition accuracy for elderly residents — under-represented in the base model's training distribution — by building a resident-specific validation set and tuning recognition thresholds against it.
- Closed 3 recurring production defect classes that passed controlled testing but failed under live traffic, by diagnosing each from structured logs and converting it into an automated pre-release check.
- Enabled product teams to ship AI features without ML expertise by exposing the recognition pipeline as 25+ secured FastAPI endpoints (JWT, RBAC, request validation) for enrollment, embedding generation, validation, and group recognition.
- Cut referral processing from 40 minutes to 7 minutes per document — an 82% reduction in staff review time — by building an AI extraction pipeline on Amazon Bedrock with a 4-member AI team.
- Ingested medical histories up to 70 pages, including scanned records, through Python/FastAPI with PyMuPDF parsing, OCR fallback, and context-aware chunking for documents exceeding model context limits.
- Auto-populated 91% of required referral fields at 94% field-level accuracy, measured against staff-verified values across 500 referral packets, by orchestrating classification, extraction, validation, and retry stages with LangGraph and enforcing a Pydantic JSON schema across demographics, diagnoses, medications, allergies, physician, and insurance fields.
- Lowered model-driven data-integrity risk in a HIPAA-regulated workflow by preventing direct LLM writes to production data: validated output passes through application authorization and persistence controls, with low-confidence fields routed to staff review.
- Instrumented model observability across the extraction service with Amazon Bedrock Guardrails and CloudWatch, tracking safety violations, invocation latency, and failure rates.
- Integrated extraction output into the existing Vue.js and Laravel referral workflow, pre-populating application forms while preserving staff approval before submission.

Research Assistant – Conversational AI & Data Systems | GitHub Aug 2024 – Dec 2025

Eastern Kentucky University, AI Research Lab *Richmond, KY*

- Designed a reusable Python/FastAPI offline evaluation harness and ran 1,000+ controlled conversational-AI experiments across 100+ prompts, comparing Llama 3.1 across local Docker/Ollama and Groq deployments using direct API, LangChain, and LangGraph architectures, with structured per-turn logging of latency, success rates, and failure reasons.
- Identified hosting environment as a larger performance factor than orchestration overhead and isolated text-to-speech as the dominant end-to-end latency contributor, at 100% conversational-turn completion.
- Increased engagement ~40% across 100+ practice sessions by building and refining a full-stack RAG-powered conversational AI sales coach (Flutter, Flask, Gemini, LangChain), tuning prompt design and retrieval relevance; 1st place and $5,000 at EKU Scholars Tank 2025.
- Co-authored an applied AI research paper documenting methodology, model benchmarking, quantitative findings, architectural trade-offs, and system limitations (in preparation).

Software Engineer – Product & Platform Engineering Jan 2022 – Aug 2024

NxtWave *Hyderabad, India*

- Owned the AI interview backend — Python/FastAPI services, session state, LLM inference, STT/TTS integration, and response evaluation — for a platform serving 50K+ learners, reducing manual interview-evaluation effort ~90%.
- Built components of the RAG personalization pipeline — chunking and embedding learner resumes and course content into the Pinecone vector database, retrieving top-k candidate-specific context per question, and carrying multi-turn state through LangChain conversation memory so follow-ups conditioned on prior answers.
- Implemented schema-constrained LLM evaluation returning structured JSON across a four-dimension rubric (introduction, communication, technical knowledge, coding), parsing and persisting per-category scores, an overall score, and generated recommendations per session.
- Improved LLM scoring consistency by replaying identical transcripts across repeated runs, increasing repeat-run score agreement to 92% through fixed scoring rubrics and deterministic decoding settings.
- Built in-browser proctoring for 10K+ interview sessions using the browser MediaDevices and Screen Capture APIs for camera and screen recording, with Page Visibility and window blur events flagging tab switches and recordings persisted for human review and audit.
- Ran and tracked 200+ experiment runs across PyTorch-based components, prompt variants, and evaluation configurations in MLflow and Weights & Biases, using paired A/B comparisons on quality, latency, and reliability to select production configurations that cut evaluation latency 25%.
- Contributed to platform performance and reliability alongside the AI work, cutting API latency ~30% during peak usage by refactoring backend functionality into independently deployable microservices and optimizing PostgreSQL, MySQL, MongoDB, and Redis access patterns.
- Cut production defects ~40% through automated testing, static analysis, CI/CD validation, structured logging, code reviews, and production-release controls.
- Containerized and deployed backend and model-serving services with Docker and Kubernetes across GCP and AWS, implementing JWT authentication, RBAC, IAM, and secrets management while supporting independent scaling during high-concurrency usage.

Full Stack Developer Intern Jun 2021 – Dec 2021
NxtWave *Hyderabad, India*

- Reduced manual content-entry effort ~60% for 50+ curriculum designers by building an internal CMS with React.js and Node.js/Express APIs.

Data Science Intern – Machine Learning Jun 2020 – Dec 2020
Cyberaegis IT Solutions *Hyderabad, India*

- Reached 94.2% facial-recognition accuracy on 10K+ images at sub-100ms inference latency by developing an end-to-end deep-learning system with Python, TensorFlow, OpenCV, CNNs, and transfer learning.
- Enabled real-time inference and efficient querying over 1M+ prediction records by integrating the trained model with Flask REST APIs and designing indexed SQL storage.

SELECTED PROJECTS

Conversational AI Architecture & Agentic Workflow Lab (ongoing) | *Python, FastAPI, LangChain, LangGraph, Hugging Face, RAG, pgvector, Ollama, Groq, Docker, STT/TTS* | GitHub 2024 – Present

- Cut prompt tokens per turn ~80% by turn 10, measured across 100 conversations on the existing per-turn logging harness, by persisting dialogue state in LangGraph rather than re-sending full history as context.
- Integrated Hugging Face Sentence Transformers into the RAG pipeline to generate document and query embeddings, enabling retrieval quality and embedding latency to be evaluated independently of the generation model.
- Reduced retrieval and cloud-model calls ~60% across the same 100-conversation set, at equivalent answer quality, by adding a lightweight Hugging Face intent router ahead of the generation layer and extending the framework with conditional routing, agentic tool-calling, and retry/fallback paths across local (Docker/Ollama) and cloud (Groq) execution.
- Recovered ~90% of failed turns across those 100 conversations — speech-recognition errors, response-parsing defects, and tool/API failures — through node-level retry and fallback routing, instrumented with per-turn logging of routing decisions, embedding-model choice, and token cost.

Evidence-Based Claim Verification System | *Python, FastAPI, LLM orchestration, information retrieval, Docker* | GitHub

- Engineered a six-stage claim-verification pipeline that decomposes text into atomic factual claims, retrieves supporting and refuting evidence from fact-check databases and the open web, and returns verdicts with citations — grounding decisions in retrieved evidence rather than in stylistic features of the source.
- Reduced API consumption ~60% through prompt-hash disk caching and difficulty-based model routing (Llama 3.3 70B for reasoning stages, 3.1 8B for classification), keeping ~40 LLM calls per document within a 1,000 requests/day free tier.
- Implemented natural-language-inference stance classification that distinguishes topically adjacent evidence from claim-relevant evidence, preventing false verdicts drawn from correct fact-checks about unrelated claims.
- Benchmarked on a stratified AVeriTeC sample, reaching 0.46 four-way verdict accuracy against a 0.30 majority-class baseline (n=37), with 1 of 37 false claims incorrectly classified as true.

Email Spam Classification | *Python, Machine Learning, NLP, scikit-learn, TF-IDF*

- Trained and evaluated a supervised NLP classification system on the Enron email corpus using TF-IDF vectorization (5,000 features) across SVM, Logistic Regression, and Decision Tree models; SVM reached 98.4% accuracy (F1 0.985) and Logistic Regression 98.2%, with per-model confusion-matrix analysis.

EDUCATION

M.S. Computer Science, Artificial Intelligence in Data Science | GPA 4.0 Aug 2024 – Dec 2025
Eastern Kentucky University, Richmond, KY
Relevant Coursework: Artificial Intelligence, Machine Learning, Big Data, Databases & Algorithms

B.Tech, Electronics & Communications Engineering | GPA 3.36 Jun 2018 – May 2022
Rajiv Gandhi University of Knowledge Technologies (RGUKT), Basar, Telangana, India

AWARDS & CERTIFICATIONS

- AWS Certified Generative AI Developer – Professional (AIP-C01) — Amazon Web Services, Aug 2026
- 1st Place, EKU Scholars Tank 2025 — $5,000 award for a full-stack RAG-powered AI sales-coaching application (EKU College of Business)
- Certificate of Achievement, CSIT AI Research Lab, Eastern Kentucky University — LLM systems research, Nov 2025

TECHNICAL SKILLS

Generative AI & ML: LLMs, RAG, Agentic AI workflows, LLM evaluation & benchmarking, prompt engineering, model fine-tuning, NLP, computer vision, deep learning, experiment design, A/B testing, MLOps, model observability, MLflow, Weights & Biases
Frameworks & Libraries: LangChain, LangGraph, FastAPI, Flask, Hugging Face, TensorFlow, PyTorch, scikit-learn, InsightFace, ArcFace, OpenCV, ONNX Runtime, PyMuPDF, Pydantic
Models & Platforms: Amazon Bedrock, Claude, Llama 3.1/3.3, Gemini, OpenAI GPT, Ollama, Groq
Languages & Web: Python, Java, JavaScript, TypeScript, SQL; Vue.js, React, Node.js/Express, Spring Boot, Laravel, REST APIs, microservices
Cloud & DevOps: AWS (Bedrock, Bedrock Guardrails, CloudWatch, EC2, S3), GCP (Cloud Run, IAM, Secret Manager), Docker, Kubernetes, DigitalOcean, Nginx, Ansible, GitHub Actions, CI/CD, Linux
Data & Practices: Vector databases (Pinecone, pgvector), PostgreSQL, MySQL, MongoDB, Redis, semantic search, pandas, NumPy; JWT, RBAC, testing, code reviews, Agile/Scrum